

Sistema de Turnos AWS - Guia de Explicacion del Proyecto

Este proyecto implementa un sistema de turnos completo utilizando una arquitectura moderna en AWS, con enfoque en Cloud y DevOps.

Objetivo

Desarrollar una aplicación Full Stack desplegada en AWS aplicando Infraestructura como Código (Terraform), contenedores Docker y servicios administrados.

Arquitectura

Internet → CloudFront → S3 (Frontend React)

↘
Application Load Balancer → ECS Fargate (FastAPI) → Amazon RDS PostgreSQL

Tecnologías

Frontend: React + Vite
Backend: FastAPI + SQLAlchemy
Base de datos: PostgreSQL
Infraestructura: Terraform
Contenedores: Docker
CI/CD: GitHub Actions

Servicios AWS utilizados

- IAM (credenciales y permisos)
- VPC, Subnets, Internet Gateway y Route Tables
- Security Groups
- Amazon ECR (repositorio Docker)
- Amazon ECS Fargate (ejecución del backend)
- Application Load Balancer
- Amazon RDS PostgreSQL
- Amazon S3 (hosting frontend)
- Amazon CloudFront (CDN)
- Amazon CloudWatch (logs)

Flujo de funcionamiento

1. El usuario accede al frontend React.
2. React consume la API FastAPI.
3. El ALB distribuye las solicitudes hacia ECS.
4. El contenedor procesa la solicitud.
5. SQLAlchemy consulta PostgreSQL en RDS.
6. La respuesta vuelve al frontend.

Infraestructura con Terraform

Se definieron recursos para la VPC, subredes, Security Groups, RDS, ECS, ALB, CloudWatch, S3 y CloudFront. Terraform permite crear y versionar toda la infraestructura mediante código.

Pipeline DevOps

GitHub → GitHub Actions → Docker Build → Push a Amazon ECR → Despliegue en ECS.
El frontend se compila y se publica en S3, invalidando la caché de CloudFront.

Problemas resueltos durante el proyecto

- Migración de SQLite a PostgreSQL.
- Configuración de variables de entorno.
- Integración con RDS.
- Configuración de ECS Fargate.
- Logs con CloudWatch.
- Despliegue mediante GitHub Actions.
- Identificación del problema de Mixed Content (HTTPS frontend vs HTTP backend).

Mejoras futuras

- HTTPS completo con ACM y dominio propio.
- AWS Secrets Manager para credenciales.
- Auto Scaling de ECS.
- Monitoreo con CloudWatch Alarms.
- Dominio personalizado con Route 53.

Conclusión

El proyecto demuestra conocimientos prácticos de desarrollo Full Stack, Docker, Terraform, CI/CD y los principales servicios de AWS para desplegar una aplicación moderna en la nube.